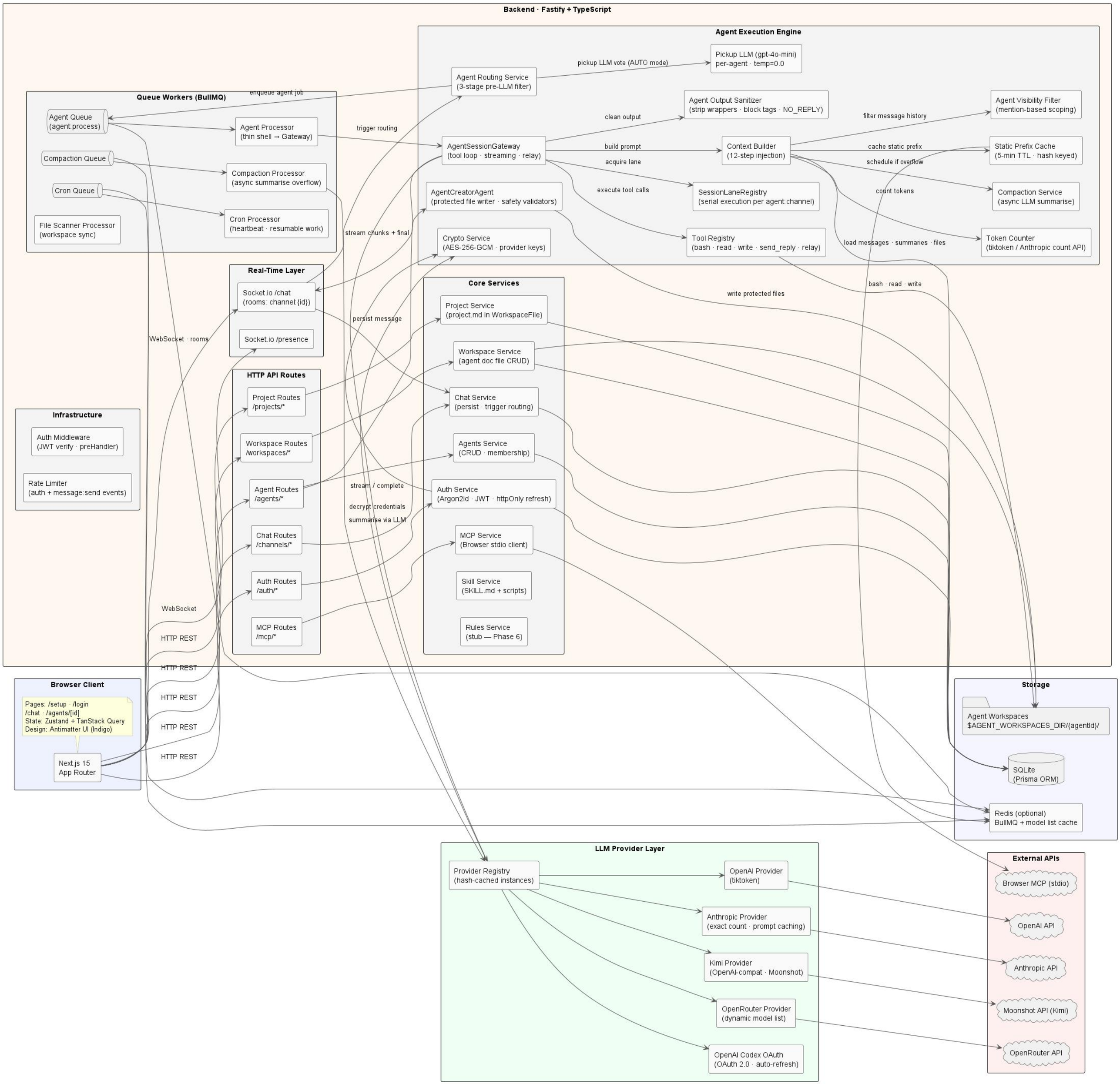
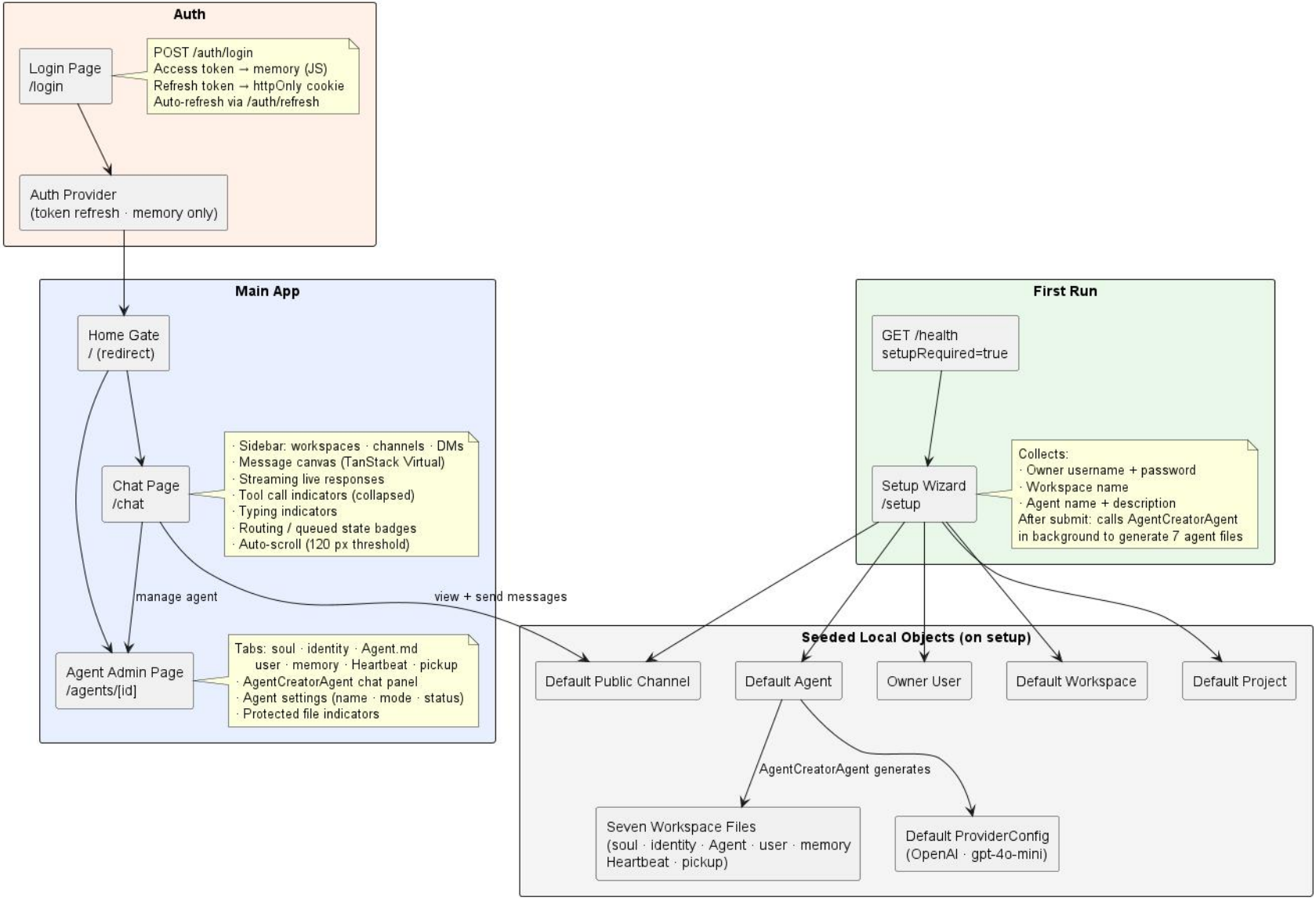
NextGenChat — System Overview

Phases 1-5 · Local-first AI agent chat platform



NextGenChat — Product Map

Pages, routes, and seeded objects



NextGenChat — Seven-File Agent Architecture

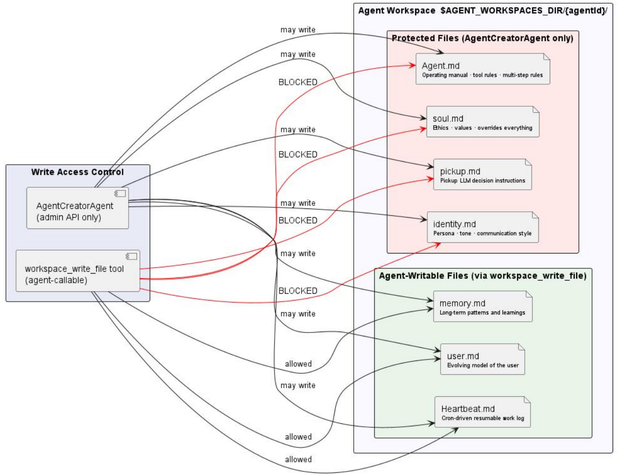
Workspace layout · write protection · context injection order

Safety Validators (before every file write)

pickup.md — must be instruction markdown, not JSON
Agent.md — must reference send_reply, memory.md, user.md, workspace_write_file
soul.md — minimum length, not a JSON blob
Any file — minimum viable length enforced
Rejected writes leave existing file untouched

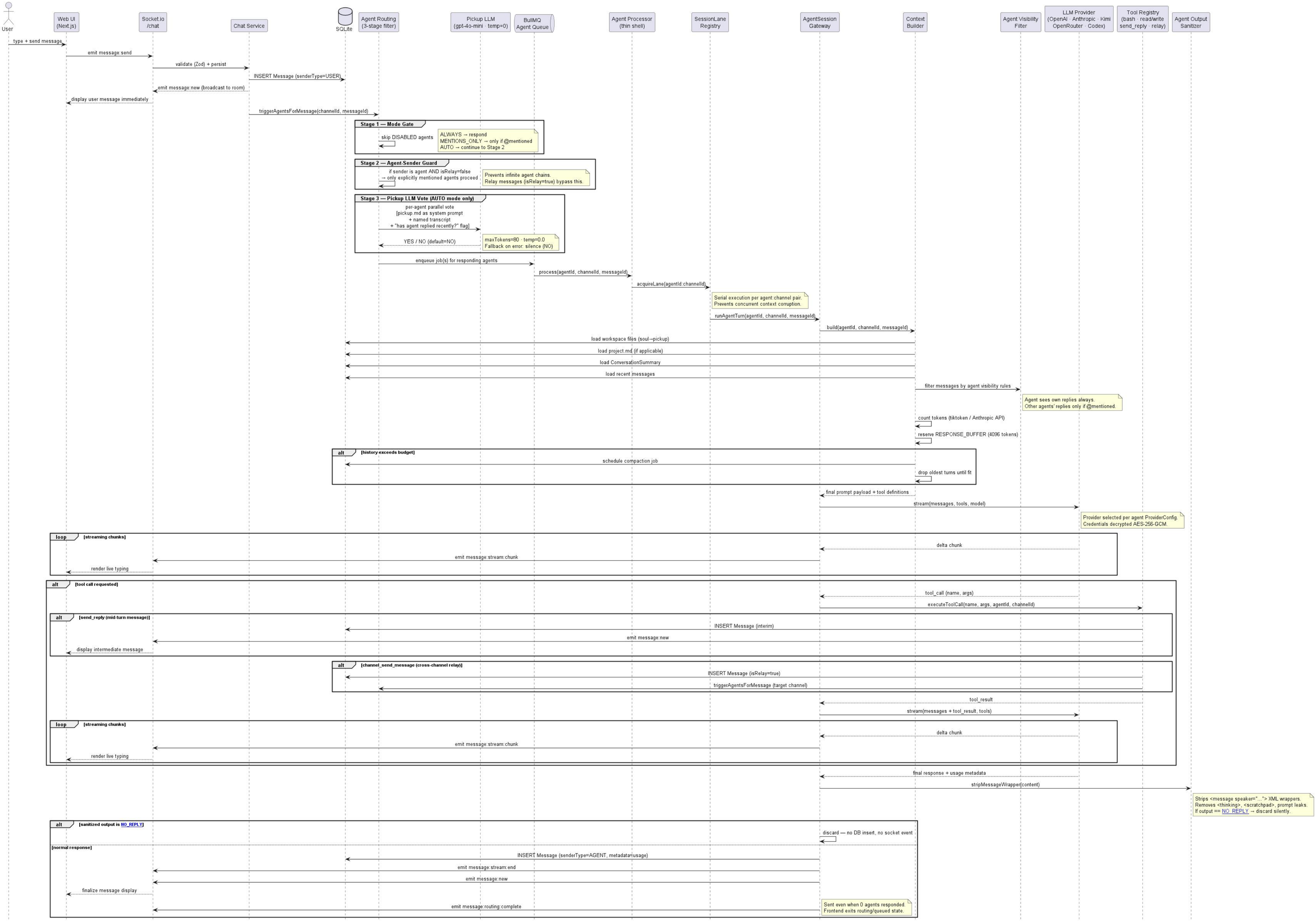
Context Injection Order (inside ContextBuilder.build)

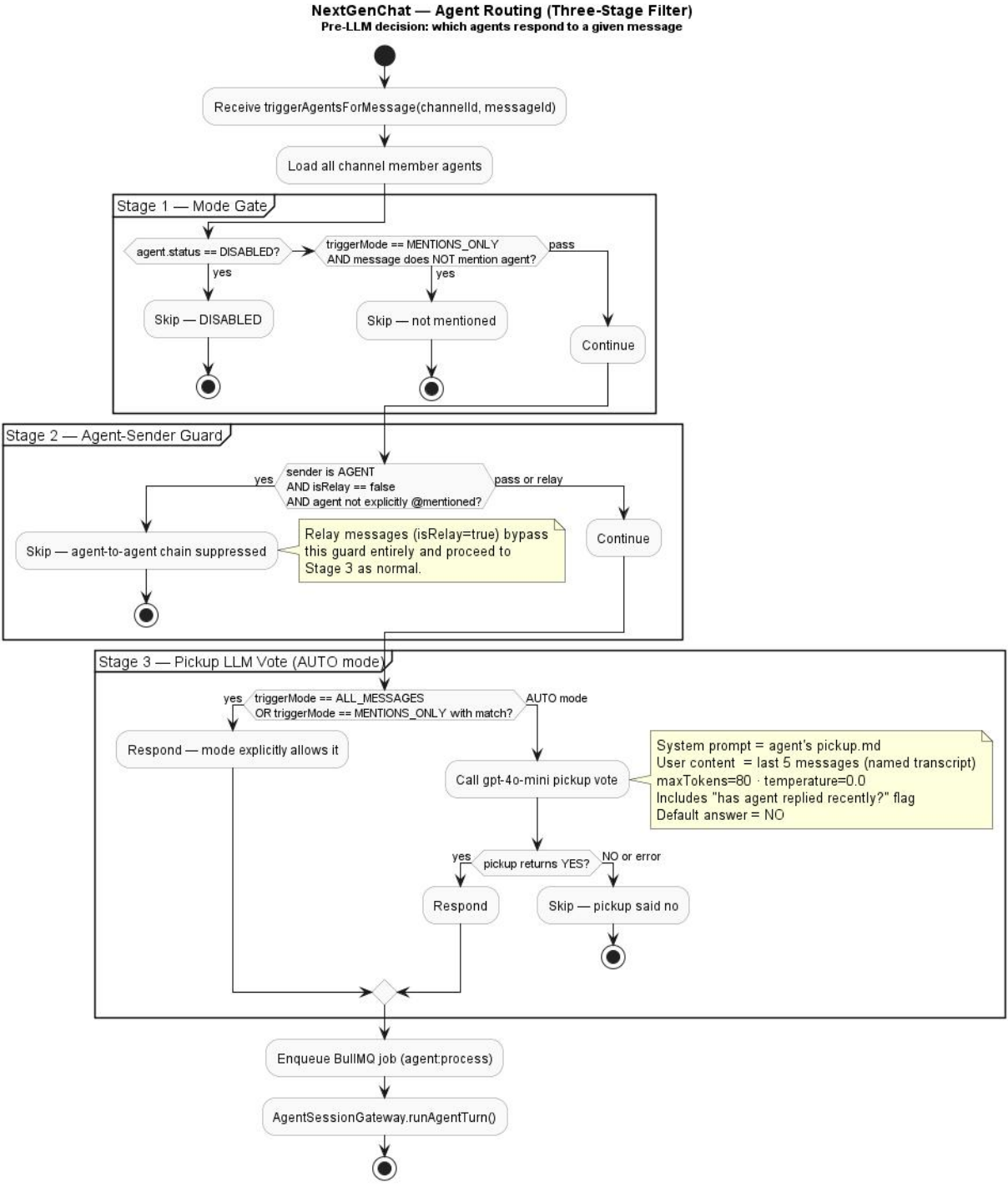
1. runtime-context.md (channel · participants · constraints)
→ 2. soul.md ← overrides everything
→ 3. identity.md
→ 4. Agent.md
→ 5. tools.md (generated · not a file)
→ 6. user.md
→ 7. memory.md
→ 8. Heartbeat.md
→ 9. project.md (if channel in project)
→ 10. agency.md (workspace-level constitution)
→ 11. Conversation summary (compacted history)
→ 12. Cross-channel context (DM channels only)
→ 13. Recent fitting messages (token budget)

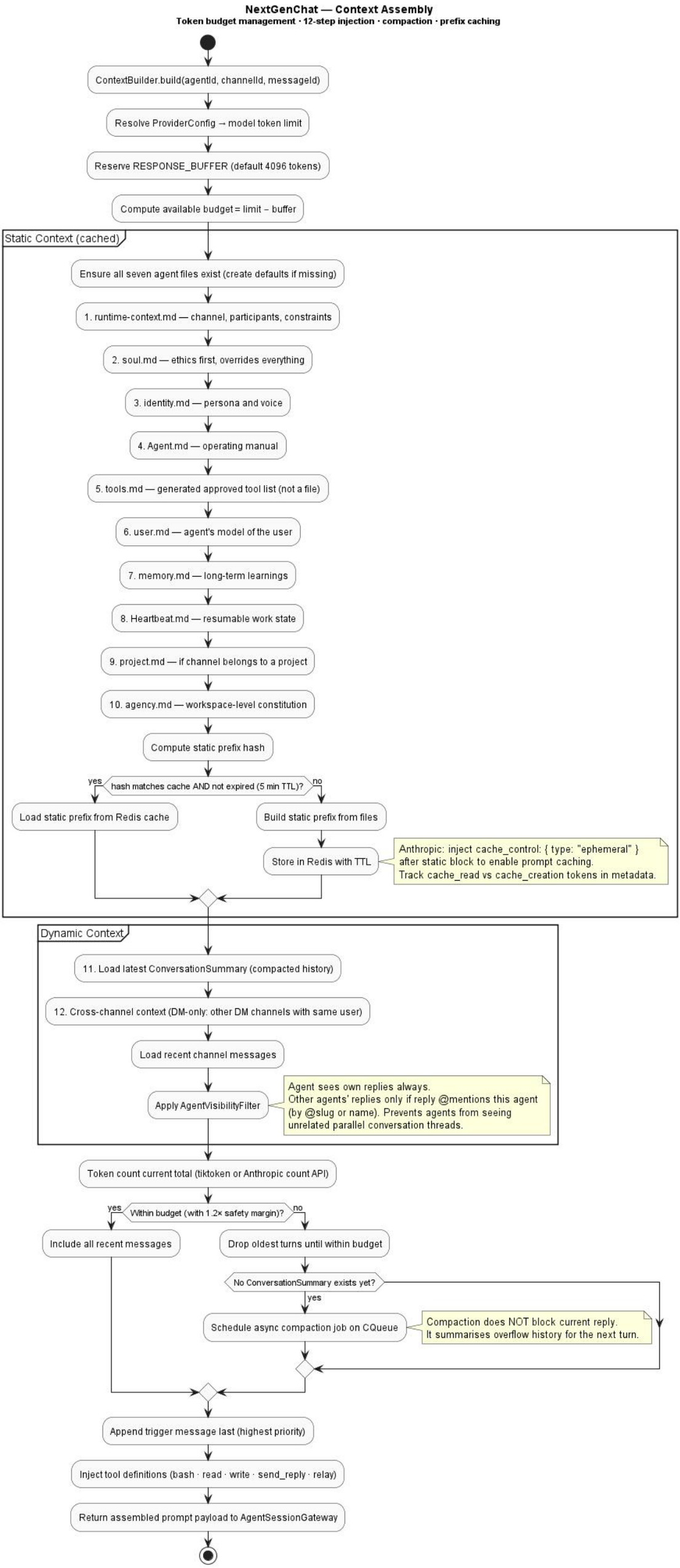


NextGenChat — Agent Message Pipeline

End-to-end path from user message to streamed reply (Phases 1-5)

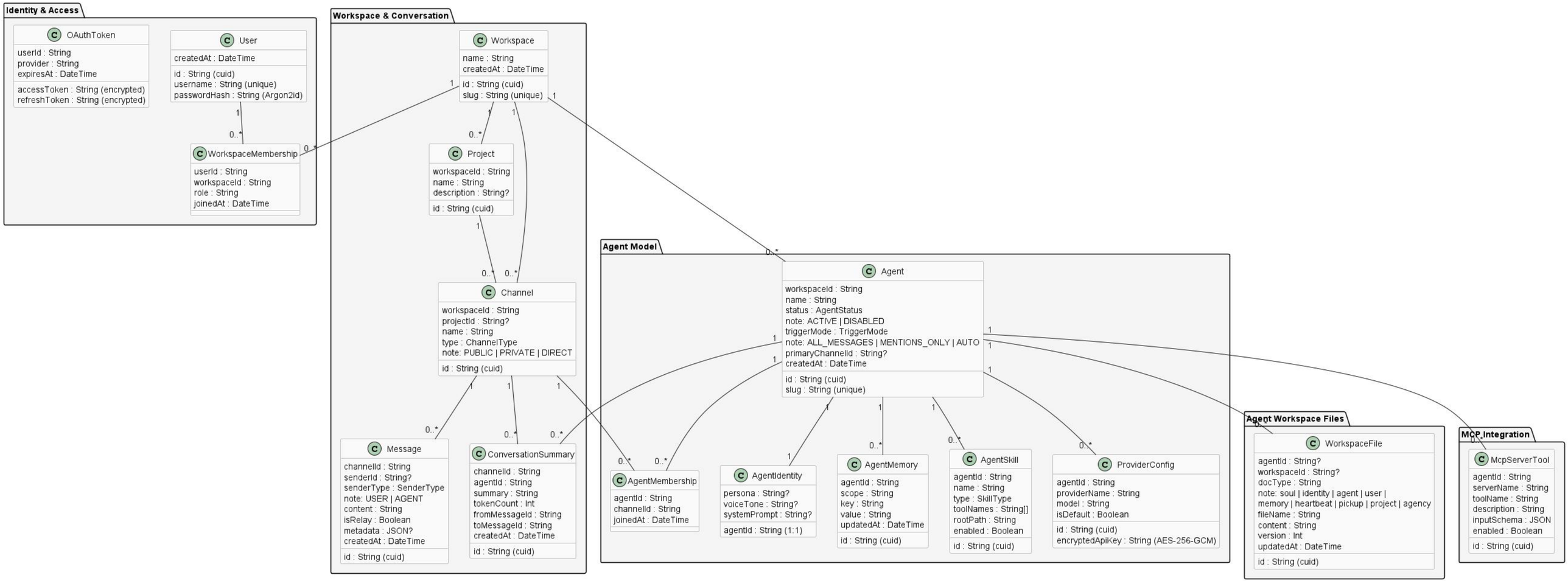


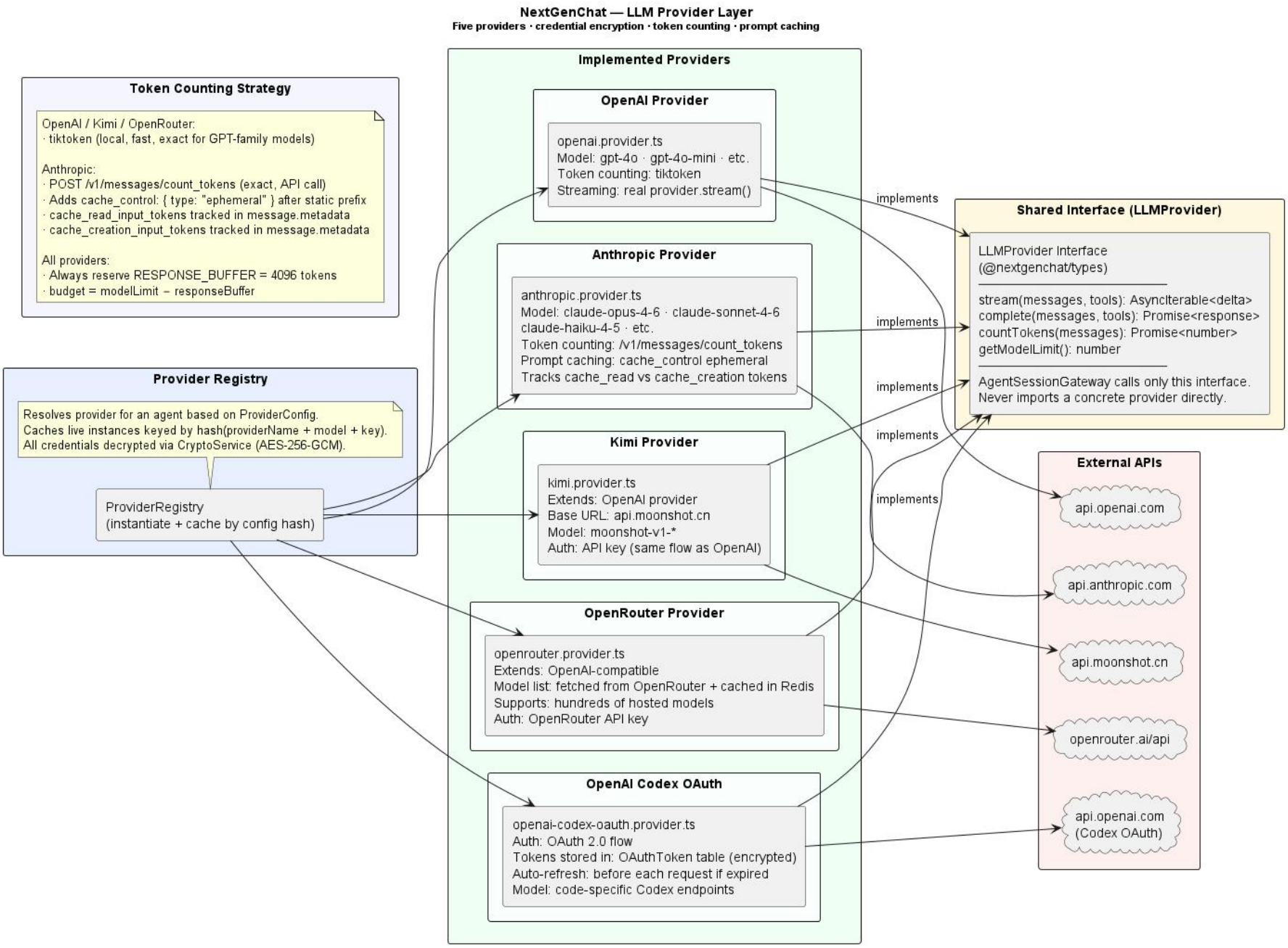

NextGenChat — Agent Routing (Three-Stage Filter)
Pre-LLM decision: which agents respond to a given message
Receive triggerAgentsForMessage(channelId, messageId)
Load all channel member agents
Stage 1 — Mode Gate
agent.status == DISABLED?
yes
Skip — DISABLED
triggerMode == MENTIONS_ONLY
AND message does NOT mention agent?
yes
Skip — not mentioned
pass
Continue
Stage 2 — Agent-Sender Guard
sender is AGENT
AND isRelay == false
AND agent not explicitly @mentioned?
yes
Skip — agent-to-agent chain suppressed
Relay messages (isRelay=true) bypass this guard entirely and proceed to Stage 3 as normal.
pass or relay
Continue
Stage 3 — Pickup LLM Vote (AUTO mode)
triggerMode == ALL_MESSAGES
OR triggerMode == MENTIONS_ONLY with match?
yes
Respond — mode explicitly allows it
AUTO mode
Call gpt-4o-mini pickup vote
System prompt = agent's pickup.md
User content = last 5 messages (named transcript)
maxTokens=80 · temperature=0.0
Includes "has agent replied recently?" flag
Default answer = NO
pickup returns YES?
yes
Respond
NO or error
Skip — pickup said no
Enqueue BullMQ job (agent:process)
AgentSessionGateway.runAgentTurn()


NextGenChat — Context Assembly
Token budget management · 12-step injection · compaction · prefix caching
ContextBuilder.build(agentId, channelId, messageId)
Resolve ProviderConfig → model token limit
Reserve RESPONSE_BUFFER (default 4096 tokens)
Compute available budget = limit − buffer
Static Context (cached)
Ensure all seven agent files exist (create defaults if missing)
1. runtime-context.md — channel, participants, constraints
2. soul.md — ethics first, overrides everything
3. identity.md — persona and voice
4. Agent.md — operating manual
5. tools.md — generated approved tool list (not a file)
6. user.md — agent's model of the user
7. memory.md — long-term learnings
8. Heartbeat.md — resumable work state
9. project.md — if channel belongs to a project
10. agency.md — workspace-level constitution
Compute static prefix hash
hash matches cache AND not expired (5 min TTL)?
yes
no
Load static prefix from Redis cache
Build static prefix from files
Store in Redis with TTL
Anthropic: inject cache_control: { type: "ephemeral" }
after static block to enable prompt caching.
Track cache_read vs cache_creation tokens in metadata.
Dynamic Context
11. Load latest ConversationSummary (compacted history)
12. Cross-channel context (DM-only: other DM channels with same user)
Load recent channel messages
Apply AgentVisibilityFilter
Agent sees own replies always.
Other agents' replies only if reply @mentions this agent
(by @slug or name). Prevents agents from seeing
unrelated parallel conversation threads.
Token count current total (tiktoken or Anthropic count API)
Within budget (with 1.2× safety margin)?
yes
no
Include all recent messages
Drop oldest turns until within budget
No ConversationSummary exists yet?
yes
Schedule async compaction job on CQueue
Compaction does NOT block current reply.
It summarises overflow history for the next turn.
Append trigger message last (highest priority)
Inject tool definitions (bash · read · write · send_reply · relay)
Return assembled prompt payload to AgentSessionGateway

NextGenChat — Domain Model

All persisted entities · Prisma / SQLite




NextGenChat — LLM Provider Layer
Five providers · credential encryption · token counting · prompt caching
Token Counting Strategy
OpenAI / Kimi / OpenRouter:
· tiktoken (local, fast, exact for GPT-family models)
Anthropic:
· POST /v1/messages/count_tokens (exact, API call)
· Adds cache_control: { type: "ephemeral" } after static prefix
· cache_read_input_tokens tracked in message.metadata
· cache_creation_input_tokens tracked in message.metadata
All providers:
· Always reserve RESPONSE_BUFFER = 4096 tokens
· budget = modelLimit − responseBuffer
Provider Registry
Resolves provider for an agent based on ProviderConfig.
Caches live instances keyed by hash(providerName + model + key).
All credentials decrypted via CryptoService (AES-256-GCM).
ProviderRegistry
(instantiate + cache by config hash)
Implemented Providers
OpenAI Provider
openai.provider.ts
Model: gpt-4o · gpt-4o-mini · etc.
Token counting: tiktoken
Streaming: real provider.stream()
Anthropic Provider
anthropic.provider.ts
Model: claude-opus-4-6 · claude-sonnet-4-6
claude-haiku-4-5 · etc.
Token counting: /v1/messages/count_tokens
Prompt caching: cache_control ephemeral
Tracks cache_read vs cache_creation tokens
Kimi Provider
kimi.provider.ts
Extends: OpenAI provider
Base URL: api.moonshot.cn
Model: moonshot-v1-*
Auth: API key (same flow as OpenAI)
OpenRouter Provider
openrouter.provider.ts
Extends: OpenAI-compatible
Model list: fetched from OpenRouter + cached in Redis
Supports: hundreds of hosted models
Auth: OpenRouter API key
OpenAI Codex OAuth
openai-codex-oauth.provider.ts
Auth: OAuth 2.0 flow
Tokens stored in: OAuthToken table (encrypted)
Auth-refresh: before each request if expired
Model: code-specific Codex endpoints
implements
implements
implements
implements
implements
Shared Interface (LLMProvider)
LLMProvider Interface
(@nextgenchat/types)
stream(messages, tools): AsyncIterable<delta>
complete(messages, tools): Promise<response>
countTokens(messages): Promise<number>
getModelLimit(): number
AgentSessionGateway calls only this interface.
Never imports a concrete provider directly.
External APIs
api.openai.com
api.anthropic.com
api.moonshot.cn
openrouter.ai/api
api.openai.com
(Codex OAuth)